Cours Typescript

Xarala
+221 77 860 60 64
contact@xarala.co

Objectifs du Cours

- Comprendre les bases de TypeScript et ses avantages par rapport à JavaScript.
- Acquérir une maîtrise de la syntaxe et des fonctionnalités de TypeScript.
- Développer des applications TypeScript pratiques.
- Appliquer les connaissances de TypeScript à des scénarios réels(NestJS, React et NextJS).

Pré-requis

- Compréhension de base de Javascript

Introduction à TypeScript

- TypeScript est un langage de programmation **fortement typé** qui s'appuie sur JavaScript, offrant de meilleurs outils à toutes les échelles.
- Développé et maintenu par **Microsoft**, TypeScript est conçu pour le développement d'applications de grande taille et se **transcompile** en JavaScript.

Caractéristiques clés

- **Typage statique** : TypeScript introduit le typage statique dans JavaScript, ce qui permet de détecter les erreurs à la compilation plutôt qu'à l'exécution.
- **Inférence de type** : Déduit(détecte) automatiquement les types, rendant votre code plus robuste sans nécessiter d'annotations étendues.
- **JavaScript avancé** : Prend en charge les fonctionnalités modernes de JavaScript (ES6 et au-delà), y compris les classes, les modules et les fonctions fléchées.
- **Support d'outils** : Meilleure prise en charge de l'éditeur avec autocomplétion, navigation et refactorisation, grâce au système de types.

Avantages de l'utilisation de TypeScript

- Détection d'erreurs
- Expérience de développement améliorée
- Scalabilité
- Compatibilité
- Communauté et support

Configuration de l'environnement TypeScript

- Pratique

Types de base (pratique)

- Types primitifs (number, string, boolean)
- Tableaux et tuples
- Énumérations
- Any, Void, Null et Undefined

Annotations de type

Les annotations de type explicites permettent aux développeurs de spécifier les types des variables, des paramètres de fonction et des valeurs de retour. Cela aide à détecter les erreurs plus tôt et rend le code plus lisible et facile à entretenir.

Exemple:

let name:string = “Ousseynou”;

Inférence de type

L'inférence de type est une fonctionnalité de TypeScript où le compilateur déduit automatiquement le type d'une variable ou d'une fonction en fonction de la valeur qui lui est affectée ou de l'instruction de retour.

Exemple:

let name = “Ousseynou”; // on n’a pas donné le type directement.

Fonctions

- Pratique

Interfaces

- Une interface en TypeScript est un moyen de définir la structure d'un objet. Elle peut inclure des propriétés et leurs types, ainsi que des méthodes.

Exemple:

```
interface User {
  name: string
  age: number
  email: string
}
const user: User = {
  name: 'Awa',
  age: 22,
  email: awa@xarala.co',
}
```

Interface vs Type

Un type peut utiliser un type primaire

On peut avoir un type qui utilise des types de bases et des unions types.

```
type DateString = string
type Id = string | number
```

Ce n'est pas possible avec les interfaces, qui ne peuvent décrire que des objets.

Une interface peut être fusionnée

Une interface peut être définie plusieurs fois et les déclarations seront alors fusionnées.

```
interface Point {
    x: number
}

interface Point {
    y: number
}

// Point aura maintenant la forme {x:number, y:number}
```

C'est impossible avec les types, qui eux ne peuvent pas être modifiés une fois déclarés.

Une interface peut être implémentée par une classe

Lorsque l'on déclare une classe, il est possible d'indiquer que l'on implémente une interface de manière explicite.

```
class TwoDimensionalPoint implements Point {
    constructor (private x: number, private y: number) {}
}
```

En savoir plus

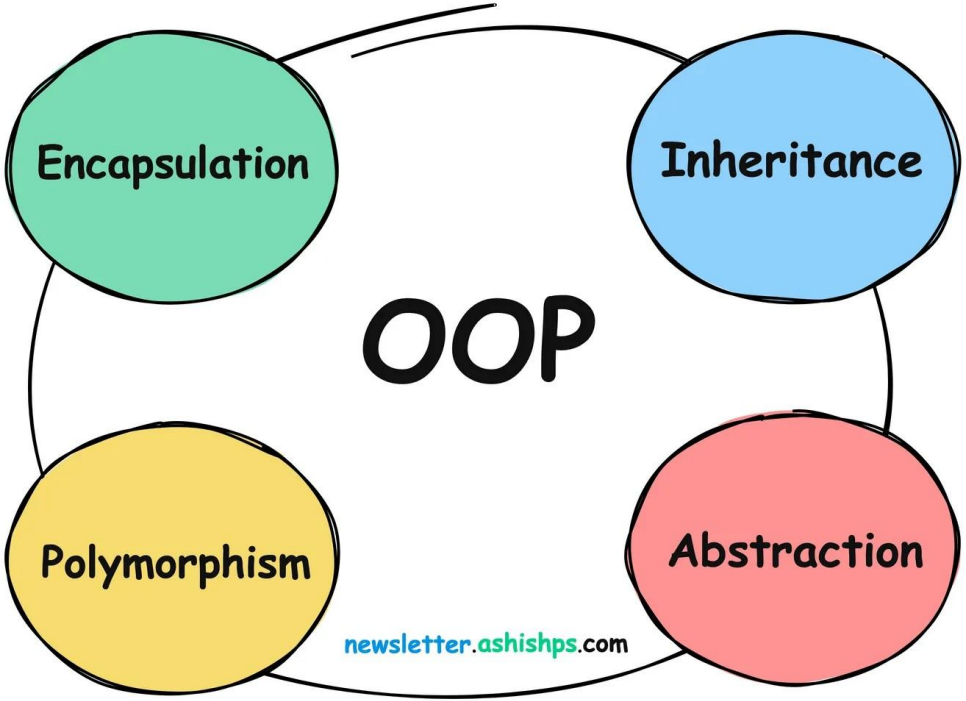
Programmation Orientée Objet (POO)

Est un modèle de programmation qui repose sur le concept de **classes** et **d'objets**.

Programmation Orientée Objet (POO)

Est un modèle de programmation qui repose sur le concept de **classes** et **d'objets**.

Concepts de bases OOP



Classes et Constructeurs

- Pratique

Héritage et mot-clé super

- Pratique

Modificateurs d'accès (public, private, protected)

- Pratique

Types Génériques

- Pratique

Modules et espaces de noms(namespaces)

- Pratique

Types avancés

- Pratique

Décorateurs

- Pratique